添加用户态指令设计专题实验报告

作者：林岳　**学号**：2024K8009929018 **专题范围**：实践任务 10、11（在五级流水 CPU 上添加用户态运算指令，以及转移与字节/半字访存指令）

> 版本：v1（2026-10-14）。已跑通：课程包 xsim 下 exp10 n1~n36、exp11 n1~n46 全 PASS。本版按课堂助教反馈补充「优化」内容并整理排版；后续将补充数据通路插图。

一、任务与判据

- **exp10（添加用户态运算指令）**：在五级流水（实践 7~9）基础上补齐用户态运算类指令——立即数、比较、逻辑、移位，以及乘除法：`pcaddu12i`、`slti`、`sltui`、`andi`、`ori`、`xori`、`sll.w`、`srl.w`、`sra.w`、`mul.w`、`mulh.w`、`mulh.wu`、`div.w`、`mod.w`、`div.wu`、`mod.wu`。测试范围 **n1~n36**。
- **exp11（转移与访存扩展）**：新增有/无符号比较分支 `blt / bge / bltu / bgeu`，与字节/半字访存 `ld.b / ld.h / ld.bu / ld.hu / st.b / st.h`。测试范围 **n1~n46**。
- **判据**（课程口径）：xsim 逐指令比对参考执行流（`gettrace/golden_trace.txt`），全过打印 `----PASS!!!`；上板成功为数码管停 `0x14000014`、双绿灯亮。

二、设计思路

2.1 乘法：ALU 内单周期实现

`alu_op` 扩展出 15/16/17 三位，乘法作为 ALU 的普通运算：`alu.v` 内计算两条 64 位积——无符号 `prod_u` 与有符号 `prod_s`；`mul.w` 取低 32 位，`mulh.w` 取有符号高 32 位，`mulh.wu` 取无符号高 32 位。综合映射到 DSP，不占额外周期。

2.2 除法：独立 `divider` 模块（多周期）

`div.w / mod.w / div.wu / mod.wu` 由独立模块 `divider` 完成，`mycpu_top` 中显式例化（接口见 §3.2）：

- 商向零截断、余数取被除数符号（有符号版本）；
- `busy` 期间冻结前段，消费者在 ID 停等；结果经 EX→MEM→ID 前递，保证不把未完成值送出去。

2.3 exp11：分支与字节/半字访存

- **分支**：ID 判出（signed / unsigned 两套比较器），命中当拍冲刷 IF / if_req 并改道，只付 2 个气泡。
- **字节/半字访存**：`ls_type` 随流水线传递（0 none / 1–3 st.b/h/w / 4–6 ld.b/h/w / 7–8 ld.bu/hu）；写通路按地址生成字节掩码与重复数据，读通路按地址选择字节/半字并做符号/零扩展。

三、课上助教反馈与对应优化

3.1 助教提出的问题

课上助教就 exp11 的代码组织提出以下问题（关键词原样记录）：

> 接口、模块、实例化、定义重复、信号空置。

具体到代码：

1. **除法没有做成独立模块**：除法器仍是 `mycpu_top` 的 EX 段里一段内联状态机，接口与实例化都不清晰；
2. **`alu_op[21:18]` 空置**：div/mod 的四位独热码在 `alu.v` 内根本无人使用（"定义了但没用到"）；
3. **定义重复**：`id_is_divmod` 与 `ex_alu_is_div = |ex_alu_op[21:18]` 是同一含义的两份写法；
4. **信号冗余**：除法相关信号定义重复、组织松散。

3.2 优化一：除法器模块化 + 显式实例化

把内联状态机抽成独立模块并实例化，接口最小化、职责单一：

```
divider u_divider(
    .clk    (clk                      ),
    .reset  (reset                    ),
    .start  (ex_div_start             ),
    .unsign (inst_div_wu | inst_mod_wu ),
    .mod    (inst_mod_w  | inst_mod_wu ),
    .a      (id_rj_value              ),
    .b      (id_rkd_value             ),
    .busy   (ex_div_busy              ),
    .result (ex_div_result            )
    );
```

原状态机（49 行）全部收进 `module divider`；顶层只剩"发 start、看 busy、用 result"三件事。

等价性验证：复跑 exp11，**n1~n46 全 PASS**，且 `$finish` 时间与改前**逐拍一致**（1419475 ns）——纯结构整改、行为零变化。

3.3 优化二：编码收窄与信号去重

- `alu_op` 由 **22 位收窄到 18 位**：删除 ALU 内部从不消费的 div/mod 四位；
- div 判定只保留**单一定义** `id_alu_is_div`，经流水线寄存器 `ex_alu_is_div` 携带到 EX 使用；删除重复写法；
- 全量"定义了但未使用"信号审计，清零。

3.4 优化三：除法算法提速

在模块化基础上把算法换掉，**结果不变、周期数大降**：

- **平凡快速路径**：除数或除数为 0、`|被除数| < |除数|` 时，accept 当拍直接出结果（`busy` 不拉高、零停顿）——原来这些也要跑满 32 拍；
- **对齐迭代**：先求除数与被除数的前导零差 `shift = clz(b) − clz(a)`，把除数左移对齐被除数的最高位，只产生"有效商位"：

  迭代数 = `clz(b) − clz(a) + 1`

  例：`1000 ÷ 3` 只需 **9 拍**（商 333、余 1）；两个随机 32 位数相除常见只有 **1~4 拍**；最坏仍为 32 拍（如除数 = 1 且被除数最高位为 1）。每拍动作缩为：

```
if (rem >= dal) begin rem <= rem - dal; quot <= quot | qbit; end
if (qbit == 32'd1) busy_r <= 1'b0;
else begin dal <= dal >> 1; qbit <= qbit >> 1; end
```

- **结构简化**：删除 64 位 `acc` 与 `step`；符号修正移到输出 mux 组合完成。

提速验证：课程包 xsim **n1~n46 全 PASS**；`$finish` **1419475 → 1234675 ns（减少约 13%）**；提速集中在除/模测试段（n30~n36：到 n36 累计 1125575 → 940775 ns）。

3.5 优化前后对照

项目	优化前（原版）	优化后（本版）
除法器位置	`mycpu_top` EX 段内联 FSM	独立 `module divider` + 显式例化
`alu_op` 宽度	22 位（含 4 位空置）	18 位（无空置）
div 判定定义	两处同义写法	单一定义 + 寄存器携带
算法	固定 32 拍 restoring	平凡快路径 + 对齐迭代（平均个位数拍）
关键寄存器	含 64 位 `acc`、`step`	无（仅 4 个 32 位数据寄存器）
exp11 仿真时间	1419475 ns	**1234675 ns（-13%）**
功能正确性	PASS	PASS（全 46 点）

四、验证与结果

实验	测试范围	结果（xsim）	日志
exp10	n1~n36	PASS，1126075 ns	`prj3/logs/exp10_xsim_run_20261012.log`
exp11	n1~n46	PASS，1419475 ns（优化前）	`prj3/logs/exp11_xsim_run_20261013_clean.log`
exp11（本版）	n1~n46	**PASS，1234675 ns（-13%）**	`prj3/logs/exp11_xsim_run_20261014_opt.log`

- patch 链：`exp9 → exp10`、`exp10 → exp11` 两段均闭环复验（应用后与目标 RTL 逐字节一致）。
- bit：超前版已出（exp10 WNS+0.925 / exp11 WNS+0.448）；课程包口径 bit 待重出。

五、调试记录

1. **"逐拍等价"**：结构整改后复跑，`$finish` 时间分毫不变（1419475 ns）——无行为变更的强证据；优化版则"功能全过 + 时间下降"，证明只提速、不改结果。
2. **边角情形检查**：对除数为 0、被除数为 0、`|a| < |b|`、最小负数等情形逐一核对商/余数，与旧版一致。

六、小结

1. 本专题在流水线上补齐了乘除法与转移/访存类指令；乘法走 ALU、除法走独立多周期单元，是"面积 / 吞吐 / 可读性"折中的结果。
2. 按助教意见完成模块化与信号清理后，又对除法器做了算法级优化：**平均迭代从 32 拍降到个位数，总仿真时间 -13%。**
3. 两次改动都做了可复核的等价性/正确性验证——结构改动"逐拍不变"，算法改动"全点通过"。